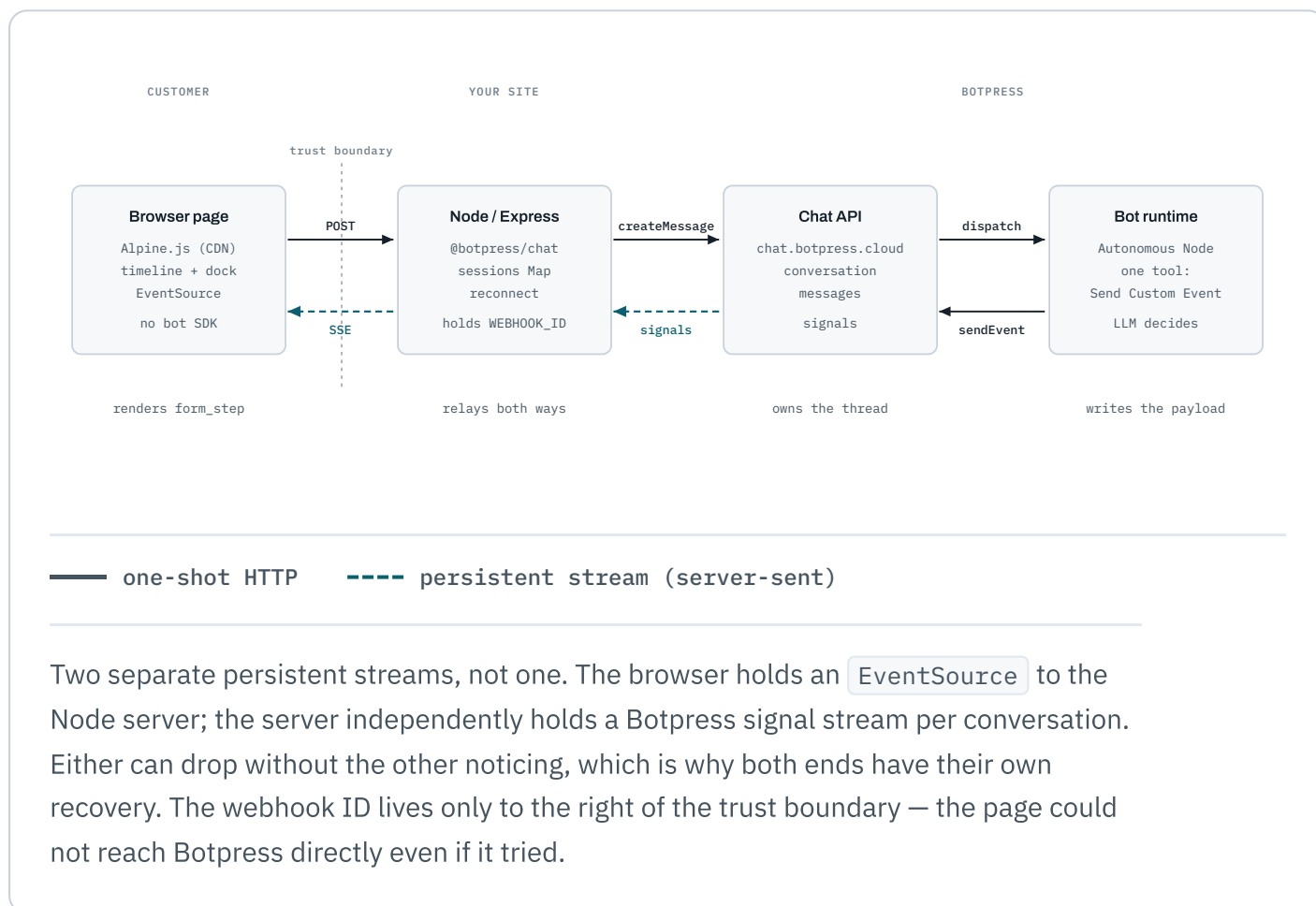


Hybrid contact form on the Botpress Chat API

A form-first page where an LLM decides what to ask next and drafts the reply. Botpress supplies the conversation and the intelligence; every pixel is the customer's own front end. This is how the four parts talk.

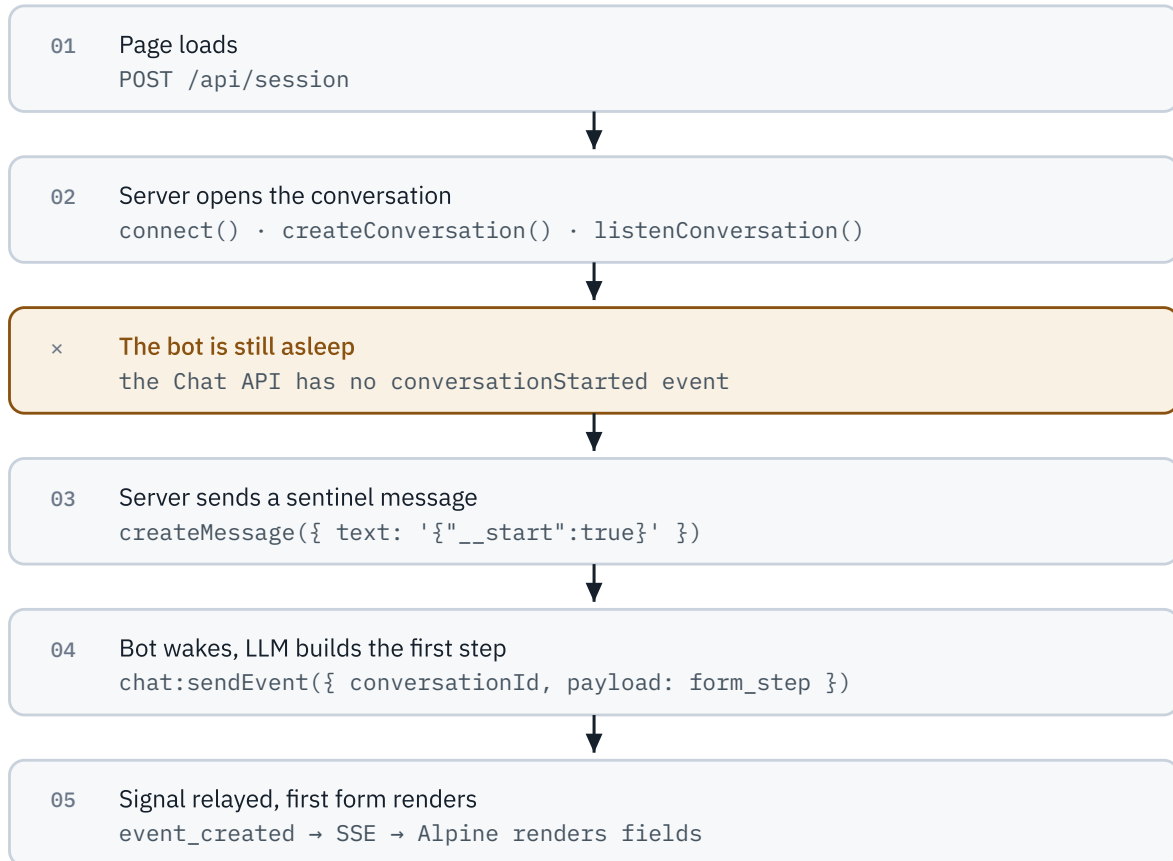
The four parts

Nothing here is a chat widget. The page holds its own UI and talks to a small Node server, which is the only party that ever holds the webhook ID or speaks to Botpress.



Waking the bot

The least obvious part of the design, and the one that will bite anyone rebuilding it. Creating a conversation does not start one.



Every later turn repeats steps 03 to 05: the page posts the customer's answers as a message, the bot replies with another `form_step`. The sentinel exists only because step 02 leaves the bot idle — with no `conversationStarted` event on the Chat API, a conversation with no message in it never reaches the bot.

What crosses the wire

One message type in each direction carries the whole interaction. The bot's payload is the page's rendering instruction; the page's payload is plain JSON in a message body.

```

{
  "type": "form_step",
  "title": "Notre réponse",
  "done": false,
  "fields": [ ... ],      // form to
render
  "answer": "...",        // drafted reply
  "links": [ ... ],        // allow-listed
only
  "choices": [ ... ]      // resolve /
escalate
}

```

```

// answers to the current form
{ "name": "...", "topic": "Annulation" }

// free-text follow-up
{ "followUp": "..." }

// the customer's decision
{ "choice": "resolved" | "human" }

```

Server endpoints

Four routes, all session-scoped. The server is a relay with a lifecycle, not an application.

ROUTE	DOES	BOTPRESS CALL
POST /api/session	Opens a conversation, starts the signal listener, sends the sentinel	connect · createConversation · listenConversation · createMessage
GET /api/stream/:id	SSE to the browser; relays bot messages and form_step events	– (relay only)
POST /api/answers/:id	Sends answers, a follow- up, or a choice as one message	createMessage
POST /api/session/:id/end	Disposes the session on restart	listener.disconnect

Things that are load-bearing

Each of these was found by breaking it. They look like details and behave like requirements.

Conversation ID must be manual

The Send Custom Event tool defaults its Conversation ID to *Let AI Decide*. Left that way the LLM emits `conversation.id`, which is undefined in the sandbox, and every call fails. Set it to Manual with `{{event.conversationId}}`.

Both streams need recovery

The Botpress `SignalListener` has a 60-second watchdog and no retry of its own — and a person filling in a form is silent far longer than that. The server reconnects; sessions with no browser attached are reaped after two minutes.

The link allow-list

The bot may only emit URLs from a fixed list in its instructions, and never a raw URL inside the answer text. Without it, plausible-looking paths get invented.

Escalation is the customer's call

The bot is forbidden from saying a request is being transferred or that an advisor will confirm. Pre-empting the second button makes the drafted answer worthless — it is the one thing the customer can already do without it.

Proof of concept — not production. Identity is anonymous auto-auth rather than member-session keys, sessions live in process memory, and the Zendesk handover is not wired. Bot and webhook identifiers are deliberately omitted from this page.